

Sistema de Gestão de Lojas Maçônicas — Documentação Funcional e Técnica

Projeto "gestao-loja" · Produção: <https://jeru.olgado.org> · Versão de 15/07/2026 · SaaS multi-tenant

1. Visão Geral

WebApp SaaS multi-tenant para gestão administrativa de Lojas Maçônicas (GOB-SP), cobrindo Secretaria, Tesouraria, documentos oficiais, assinaturas digitais e comunicação institucional. Cada Loja é um tenant isolado: todas as tabelas possuem `lojaId` e os dados de uma oficina jamais vazam para outra.

Perfis e Governança

- **Login:** CIM (Cadastro de Identificação Maçônica) + CPF + senha, com validação de e-mail. Recuperação de senha com 2FA por e-mail (código de 6 dígitos, 15 min, 5 tentativas).
- **Perfis:** Venerável Mestre, Secretário, Tesoureiro, Conselho de Contas (somente leitura nos módulos executivos), Obreiro e cargos de Loja completos (1º/2º Vigilante, 1º/2º Diácono, Orador, Guardas, Diretor de Cerimônias).
- **Cargos do rito personalizados:** o Secretário cadastra cargos conforme o rito da Loja (`/secretaria/cargos`), usados na nomeação e no histórico dos membros.

2. Stack Tecnológica

Camada	Tecnologia
Front-end	Next.js 16 (App Router), React 19, Tailwind CSS 4, Shadcn UI, dnd-kit (Kanban drag-and-drop)
Back-end	Next.js API Routes, NextAuth v5 (autenticação), Zod (validação)
Banco de dados	PostgreSQL + Prisma ORM (20 models)
PDF / Documentos	pdf-lib + fontkit (geração), @signpdf (validação PAdES), jszip (DOCX/PPTX), qrcode
Integrações	Google Drive API (OAuth por loja, escopo drive.file), Gmail/nodemailer (e-mails da loja), gateway Asaas (cobranças), assinador.iti.br (assinatura gov.br)

3. Módulo Secretaria

3.1 Dashboard (Gestão à Vista)

- **To-Do de Assinaturas:** cards de Atas, Quitte Placets e Pranchas aguardando assinatura digital dupla, com botão "Assinar agora" (modal com pré-visualização e confirmação por senha, com trilha de auditoria).
- **Alertas de prazos:** interstícios atingidos (obreiros aptos a elevação/exaltação), prazo de 15 dias pós-Sessão Magna para comunicação ao portal "Sua Sessão no GOB-SP" (card fica vermelho se vencido) e auditoria cadastral (obreiros com cadastro incompleto).
- **Central de notificações:** sino no header com badge de não lidas, painel agrupado por tipo (assinatura pendente, prazo, dados faltantes, aprovação financeira), geração automática por rotina.

3.2 Pipeline de Admissão (Kanban)

Fluxo do candidato profano: **Pré-Cadastro** (antecedentes e dados civis, foto do candidato no card) → **Editais** de Consulta → **Sindicância** (comissão sigilosa e relatórios) → **Escrutínio** (votação secreta) → **Placet de Iniciação** → Iniciado/Reprovado.

3.3 Pipeline de Progressão de Graus (Kanban)

Aprendiz → Companheiro → Mestre, com travas oficiais:

Etapa	Regra
Interstício	Cálculo automático do tempo no grau; card travado até o prazo legal (Aprendiz→Companheiro: 12 meses; Companheiro→Mestre: 6 meses).
Instrução e Frequência	Validação automática no Livro de Presenças.
Exame de Proficiência	Agendamento/aprovação da avaliação ritualística.
Escrutínio	Votação em plenário.
Aguardando Placet	Prancha gerada automaticamente à Guarda dos Selos; cerimônia bloqueada até <code>placetDeferido = true</code> .
Aguardando Cerimônia	Agendamento da Sessão Magna.
Comunicação pós-cerimônia	Contagem regressiva de 15 dias para envio da matéria e fotos; gera solicitação de diplomas e patentes.
Grau Concedido	Atualiza o grau no banco e encerra o processo.

3.4 Movimentação (Quitte Placet)

- **Trava financeira:** emissão do desligamento só com `quitacaoFinanceira = true` (Nada Consta da Tesouraria).
- **Dupla assinatura:** Venerável Mestre + Secretário.

3.5 Membros

- Quadro de Obreiros com cargos, graus e status (ativo/irregular/quite).
- Histórico de graus e cargos editável, com recálculo automático do grau/cargo atual.
- Troca de senha obrigatória no primeiro acesso (senha provisória gerada pelo Secretário).
- Assinatura visual: upload da imagem da assinatura (até 500 KB) para VM, Secretário e Orador.

4. Atas, Presença e Documentos

4.1 Livro de Presença e Certificado de Visita

- Check-in digital com QR Code para membros e visitantes (nome e potência de origem).
- Visitante que informa e-mail recebe automaticamente certificado de visita em PDF; template personalizável por loja (upload de PPTX com marcadores).

4.2 Lavratura e Ciclo de Vida da Ata (Balaústre)

- **Ata pré-preenchida:** formulário com os campos livres do modelo (pauta, levantamentos, ausências, encerramento) gera o rascunho do modelo institucional — data/hora por extenso, grau/tipo da sessão, cargos identificados entre os presentes, visitantes e contagem de obreiros.
- **Fluxo:** Rascunho → minuta por e-mail (cco) a todos os irmãos ativos para validação → ajustes registrados e aplicados → liberação para assinaturas → VM assina primeiro, Secretário na sequência (trava de ordem) → envio da versão final aos irmãos do quadro.
- **Assinatura exclusiva:** na liberação o Secretário escolhe assinatura normal (interna) **ou** gov.br — nunca as duas.
- **Fluxo gov.br (assinador.iti.br):** VM baixa o PDF, assina externamente e sobe; Secretário baixa a versão com a assinatura do VM, assina e sobe o arquivo final, validado (PADES /ByteRange). A 2ª assinatura sela a ata. Código da API oficial do ITI pronto e dormante (credenciamento restrito a órgãos públicos).

- **Arquivamento:** na 2ª assinatura o PDF final vai automaticamente ao Google Drive da Loja; PDF e preview com cabeçalho institucional (logo, títulos, filete vermelho, divisa no rodapé — configurável por loja).

4.3 Pranchas e Formulários Oficiais GOB-SP

- Expedição de ofícios numerados sequencialmente por e-mail (Gmail da loja) à Guarda dos Selos (gselos@gobsp.org.br).
- Catálogo com 31 formulários oficiais (Form. 009 a 129 + atestado), por categoria, com download atrás do login.
- **Preenchimento automático:** diálogo "Preencher" gera o DOCX oficial com loja/oriente/datas/cargos (com CIM)/obreiro/candidato.
- **Assinatura gov.br do anexo:** prancha com anexo só é expedida após o Secretário assinar o anexo no assinador.iti.br e subir o PDF assinado (validação PAdES); a versão assinada é arquivada no Drive e segue anexada ao e-mail.

5. Módulo Tesouraria

- **Gateway Asaas:** cobrança recorrente por cartão/boleto (assinaturas mensais), link de cobrança avulso por capitação e webhook com baixa automática (/api/webhooks/asaas , token por loja). Configuração por loja; aprovado em sandbox.
- **Inadimplência automática:** limite configurável por loja (default 3 capitações vencidas) muda o membro para IRREGULAR; quite reverte para ATIVO.
- **Controles:** faturas, despesas, transações, doações e eventos beneficentes (models próprios).
- **Relatórios CSV:** balancete mensal, situação de mensalidades e frequência anual por membro (separador ; e BOM para Excel BR).

6. Modelo de Dados (Prisma — 20 models)

Domínio	Models
Núcleo	Lodge , User , DegreeHistory , RoleHistory , CargoRito , Notification
Sessões	LodgeSession , Attendance , Instrucao
Documentos	Ata , Prancha , Document
Financeiro	Invoice , Expense , Transaction , Donation , CharityEvent
Processos	ProcessoAdmissao , QuittePlacet , ProcessoProgressao

Campos de trava relevantes: placetDeferido , comunicadoEnviado , quitacaoFinanceira , signedByMaster/signedBySec , govbrSolicitado/govbrMasterAt/govbrSecAt , mustChangePassword , limiteInadimplencia .

7. Segurança e Conformidade

- Isolamento rigoroso por lodgeId em todas as consultas; Conselho de Contas sem permissão de escrita.
- Senhas e códigos 2FA com bcrypt; documentos assinados ficam imutáveis (trancados) com trilha de auditoria (quem, quando, IP).
- Validação PAdES (/ByteRange) dos PDFs assinados no gov.br antes de aceitar o upload.
- Layout responsivo mobile (tabelas e Kanbans com rolagem interna, grids empilhados).

8. Estado Atual e Próximos Passos

Em produção (<https://jeru.olgado.org>) com usuários reais; todo o escopo funcional acima está implementado e testado.

Frentes planejadas: painel do super admin com status de pagamento das licenças (webhook Asaas da plataforma), lembretes automáticos de instruções/interstício por e-mail e publicação do app OAuth do Google (saída do modo de teste). Pendências de configuração: chave Asaas de produção e credenciais Google OAuth definitivas.

Anexo A — Schema Prisma completo (prisma/schema.prisma)

```
// Schema – SaaS Multi-tenant para Gestão de Lojas Maçônicas
// Toda entidade de negócio carrega lodgeId (tenant) e todas as
// queries da aplicação DEVEM filtrar por ele.

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// ----- Tenant -----

model Lodge {
  id          String   @id @default(cuid())
  name        String
  number      String   @unique // nº da Loja na potência
  potencia    String?  // GOB, GLESP, COMAB etc.
  oriente     String?  // cidade/UF
  address     String?  // endereço da sede (citado no texto das atas)
  driveFolderId String? // pasta raiz da Loja no Google Drive
  pixKey      String?  // chave Pix da Loja (recebimento das capitações)
  logoUrl     String?  // logo da Loja (data URI ou URL)
  // Cabeçalho institucional das atas em PDF (linhas centralizadas sob o
  // nome da Loja, ex.: título honorífico, fundação, federação) e divisa
  // exibida no rodapé de cada página
  ataCabeçalho String?
  ataDivisa    String?
  // Frequência mínima (%) nas sessões durante o processo de progressão
  // (trava da coluna Instrução e Frequência – ajuste conforme o regulamento)
  minFreqProgressao Int @default(50)
  // Nº de capitações vencidas para o membro ficar IRREGULAR (inadimplência)
  limiteInadimplencia Int @default(3)
  // Licença do sistema cobrada da loja (opcional, definida no cadastro pelo
  // SUPER_ADMIN; a cobrança boleto/Pix sai da conta Asaas da plataforma)
  licencaValor      Decimal? @db.Decimal(10, 2)
  licencaChargeId   String?
  licencaInvoiceUrl String?
  // Instruções exigidas para progressão de grau (0 = sem exigência):
  // Aprendizes instruídos pelo 2º Vigilante, Companheiros pelo 1º Vigilante
  instrucoesAprendiz Int @default(0)
  instrucoesCompanheiro Int @default(0)
  // Certificado de Visita personalizado (upload de PPTX na config da loja):
  // fundo renderizado em PDF + posições dos placeholders; null = template padrão
  certFundoPdf      Bytes?
  certLayout        Json?
  // Gateway de pagamento Asaas (cobrança recorrente cartão/boleto)
  asaasApiKey       String? // API key da conta Asaas da Loja
  asaasWebhookToken String? // token conferido no header asaas-access-token do webhook
  googleRefreshToken String? // OAuth do Google da Loja (conexão ao Drive)
  googleEmail       String? // conta Google conectada
  createdAt         DateTime @default(now())
  updatedAt         DateTime @updatedAt

  users          User[]
  degreeHistory  DegreeHistory[]
  roleHistory    RoleHistory[]
  sessions       LodgeSession[]
  attendances    Attendance[]
  instrucoes     Instrucao[]
  atas           Ata[]
  pranchas       Prancha[]
  documents      Document[]
  invoices       Invoice[]
  expenses       Expense[]
  transactions   Transaction[]
  donations      Donation[]
  events         CharityEvent[]
  processosAdmissao ProcessoAdmissao[]
  processosProgressao ProcessoProgressao[]
  quittePlacets  QuittePlacet[]
  notifications  Notification[]
  cargosRito     CargoRito[]

  @@map("lodges")
}

// ----- Usuários -----
```

```

enum Degree {
  APRENDIZ
  COMPANHEIRO
  MESTRE
}

enum Role {
  MEMBER // Obreiro comum
  VENERAVEL_MESTRE
  SECRETARIO
  TESOUREIRO
  CONSELHO_CONTAS // somente leitura em Secretaria/Tesouraria
  // Cargos de Loja sem permissão especial no sistema (equivalem a MEMBER)
  PRIMEIRO_VIGILANTE
  SEGUNDO_VIGILANTE
  PRIMEIRO_DIACONO
  SEGUNDO_DIACONO
  ORADOR
  GUARDA_INTERNO
  GUARDA_EXTERNO
  DIRETOR_CERIMONIAS
  SUPER_ADMIN // admin master do SaaS – cria e administra lojas (tenants)
}

enum MemberStatus {
  ATIVO
  IRREGULAR // inadimplente
  LICENCIADO
  EX_MEMBRO
}

model User {
  id String @id @default(cuid())
  lodgeId String
  cim String @unique // Cadastro de Identificação Maçônica
  cpf String @unique
  name String
  email String @unique
  phone String?
  address String?
  birthDate DateTime?
  profession String?
  passwordHash String
  // Força troca de senha no primeiro acesso (senha provisória = CPF)
  mustChangePassword Boolean @default(false)
  // Recuperação de senha: código 2FA enviado por e-mail (hash bcrypt)
  resetCodeHash String?
  resetCodeExpiresAt DateTime?
  resetCodeAttempts Int @default(0)
  // Gateway Asaas: cliente e assinatura recorrente da capitação
  asaasCustomerId String?
  asaasSubscriptionId String?
  degree Degree @default(APRENDIZ)
  currentRole Role @default(MEMBER)
  // Cargo do rito (personalizado pela Loja) – exibição; o nível de acesso
  // continua sendo o de currentRole (cargos do rito = nível Obreiro)
  cargoRito String?
  // Imagem da assinatura (data URI) – VM, Secretário e Orador, para assinar atas e documentos
  signatureUrl String?
  // Foto do membro (data URI, upload pelo próprio usuário em /dashboard/perfil)
  photoUrl String?
  status MemberStatus @default(ATIVO)
  initiationDate DateTime? // data de iniciação (base do interstício)

  // Painel de Privacidade (LGPD): visibilidade de contatos aos demais membros
  isDataPublic Boolean @default(false)
  showEmail Boolean @default(false)
  showPhone Boolean @default(false)
  showAddress Boolean @default(false)
  showBirthDate Boolean @default(false)

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  lodge Lodge @relation(fields: [lodgeId], references: [id])
  degreeHistory DegreeHistory[]
  roleHistory RoleHistory[]
  attendances Attendance[]
  invoices Invoice[]
  donations Donation[]
  atasAsMaster Ata[] @relation("AtaMaster")
  atasAsSec Ata[] @relation("AtaSecretary")
  expensesAsMaster Expense[] @relation("ExpenseMaster")
  expensesAsTreasurer Expense[] @relation("ExpenseTreasurer")
  documentsUploaded Document[]
  quittePlacets QuittePlacet[]
  quittePlacetsAsMaster QuittePlacet[] @relation("QuittePlacetMaster")
  quittePlacetsAsSec QuittePlacet[] @relation("QuittePlacetSecretary")

```

```

    notifications      Notification[]
    instrucoesRecebidas Instrucao[]      @relation("InstrucaoUser")
    instrucoesRegistradas Instrucao[]    @relation("InstrucaoRegistrador")
    processosProgressao ProcessoProgressao[]

    @@index([lodgeId])
    @map("users")
}

// Evolução de graus com controle de interstício
model DegreeHistory {
    id      String      @id @default(cuid())
    lodgeId String
    userId  String
    degree  Degree
    date    DateTime // data da elevação/exaltação/iniciação
    ataId   String? // ata da sessão em que ocorreu
    createdAt DateTime @default(now())

    lodge Lodge @relation(fields: [lodgeId], references: [id])
    user  User  @relation(fields: [userId], references: [id])

    @@index([lodgeId])
    @@index([userId])
    @map("degree_history")
}

// Histórico cronológico de cargos
model RoleHistory {
    id      String      @id @default(cuid())
    lodgeId String
    userId  String
    role    Role
    cargoRito String? // nome do cargo do rito quando a nomeação for personalizada
    startDate DateTime
    endDate  DateTime?
    createdAt DateTime @default(now())

    lodge Lodge @relation(fields: [lodgeId], references: [id])
    user  User  @relation(fields: [userId], references: [id])

    @@index([lodgeId])
    @@index([userId])
    @map("role_history")
}

// Cargos personalizados da Loja conforme o rito (ex.: Hospitaleiro,
// Mestre de Harmonia, Porta-Estandarte) – cadastrados pelo Secretário
model CargoRito {
    id      String      @id @default(cuid())
    lodgeId String
    nome    String
    createdAt DateTime @default(now())

    lodge Lodge @relation(fields: [lodgeId], references: [id])

    @@unique([lodgeId, nome])
    @map("cargos_rito")
}

// ----- Sessões, Presenças e Atas -----

enum SessionType {
    ORDINARIA
    MAGNA
    ECONOMICA
    BRANCA
}

model LodgeSession {
    id      String      @id @default(cuid())
    lodgeId String
    date    DateTime
    type    SessionType @default(ORDINARIA)
    degree  Degree       @default(APRENDIZ) // grau em que a sessão foi aberta
    qrToken String       @unique @default(cuid()) // check-in via QR Code
    createdAt DateTime @default(now())

    lodge      Lodge      @relation(fields: [lodgeId], references: [id])
    attendances Attendance[]
    ata        Ata?

    @@index([lodgeId])
    @map("lodge_sessions")
}

// Livro de Presenças (membros e visitantes)
model Attendance {

```

```

id          String    @id @default(cuid())
lodgeId     String
sessionId   String
userId      String? // null = visitante
// dados do visitante
visitorName String?
visitorEmail String? // para envio do certificado de visita em PDF
visitorCim  String?
visitorLodge String? // loja de origem
visitorPotencia String? // potência de origem
checkedInAt DateTime @default(now())
viaQrCode   Boolean   @default(false)

lodge      Lodge      @relation(fields: [lodgeId], references: [id])
session    LodgeSession @relation(fields: [sessionId], references: [id])
user       User?      @relation(fields: [userId], references: [id])

@@unique([sessionId, userId])
@@index([lodgeId])
@@map("attendances")
}

// Instruções de grau ministradas pelos Vigilantes (check-in dos instruídos):
// Aprendizizes com o 2º Vigilante, Companheiros com o 1º Vigilante
model Instrucao {
  id          String    @id @default(cuid())
  lodgeId     String
  userId      String // obreiro instruído
  degree      Degree // grau da instrução
  date        DateTime
  tema        String?
  registradaPorId String
  createdAt   DateTime @default(now())

  lodge      Lodge @relation(fields: [lodgeId], references: [id])
  user       User  @relation("InstrucaoUser", fields: [userId], references: [id])
  registradaPor User @relation("InstrucaoRegistrador", fields: [registradaPorId], references: [id])

  @@index([lodgeId])
  @@index([userId])
  @@map("instrucoes")
}

enum AtaStatus {
  RASCUNHO
  EM_VALIDACAO // enviada aos irmãos para validação antes das assinaturas
  AGUARDANDO_ASSINATURAS
  ASSINADA // VM + Secretário assinaram – imutável
}

// Ata (Balaústre) – trava: exige assinatura conjunta de VM e Secretário
model Ata {
  id          String    @id @default(cuid())
  lodgeId     String
  sessionId   String    @unique
  number      Int // numeração sequencial por loja
  content     String
  status      AtaStatus @default(RASCUNHO)

  signedByMasterId String?
  signedByMasterAt  DateTime?
  signedBySecId     String?
  signedBySecAt     DateTime?

  driveFileId String? // PDF assinado no Google Drive

  // Assinatura digital gov.br (PADES) – cada assinante assina com a própria conta
  govbrPdf Bytes? // PDF com as assinaturas digitais acumuladas
  govbrMasterAt DateTime? // assinatura gov.br do Venerável Mestre
  govbrSecAt DateTime? // assinatura gov.br do Secretário
  govbrUploadedAt DateTime? // upload do PDF assinado externamente no assinador.iti.br
  govbrSolicitado Boolean @default(false) // Secretário optou por encaminhar ao gov.br
  ajustes String? // ajustes solicitados pelos irmãos na validação
  sentForReviewAt DateTime? // envio aos irmãos para validação (antes das assinaturas)
  sentToMembersAt DateTime? // envio da versão final assinada aos irmãos
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  lodge      Lodge      @relation(fields: [lodgeId], references: [id])
  session    LodgeSession @relation(fields: [sessionId], references: [id])
  signedByMaster User?    @relation("AtaMaster", fields: [signedByMasterId], references: [id])
  signedBySec   User?    @relation("AtaSecretary", fields: [signedBySecId], references: [id])

  @@unique([lodgeId, number])
  @@map("atas")
}

// _____ Documentos e Pranchas _____

```

```

// Correspondência expedida com numeração sequencial automática
model Prancha {
  id          String    @id @default(cuid())
  lodgeId     String
  number      Int // sequencial por loja/ano
  year        Int
  subject     String
  recipient   String
  content     String
  driveFileId String?
  govbrPdf    Bytes?    // anexo assinado no assinador.iti.br
  govbrSignedAt DateTime? // upload do anexo assinado (gov.br)
  createdAt   DateTime @default(now())

  lodge Lodge @relation(fields: [lodgeId], references: [id])

  @@unique([lodgeId, year, number])
  @map("pranchas")
}

enum DocumentType {
  ATA_ESCANEADA
  HISTORICO
  REGULAMENTO
  FINANCEIRO
  OUTRO
}

model Document {
  id          String    @id @default(cuid())
  lodgeId     String
  uploadedById String
  title       String
  type        DocumentType @default(OUTRO)
  driveFileId String // arquivo no Google Drive da Loja
  mimeType    String?
  sizeBytes   Int?
  createdAt   DateTime @default(now())

  lodge      Lodge @relation(fields: [lodgeId], references: [id])
  uploadedBy User @relation(fields: [uploadedById], references: [id])

  @@index([lodgeId])
  @map("documents")
}

// ----- Tesouraria -----

enum InvoiceStatus {
  PENDENTE
  PAGA
  VENCIDA
  CANCELADA
}

enum PaymentMethod {
  PIX
  CARTAO
  BOLETO
  DINHEIRO
  MANUAL
}

// Mensalidade (capitação) gerada automaticamente para membros ativos
model Invoice {
  id          String    @id @default(cuid())
  lodgeId     String
  userId      String
  description  String // ex.: "Capitação 07/2026"
  referenceMonth Int
  referenceYear Int
  amountCents Int
  dueDate     DateTime
  status      InvoiceStatus @default(PENDENTE)

  // Pix dinâmico
  pixTxid     String?    @unique // id da cobrança no PSP (chave do webhook)
  pixCopiaECola String?
  paidAt      DateTime?
  paidMethod   PaymentMethod?
  gatewayChargeId String? @unique // id do pagamento no gateway (chave do webhook Asaas)
  gatewayInvoiceUrl String? // link de pagamento (boleto/cartão/Pix) no gateway

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  lodge      Lodge @relation(fields: [lodgeId], references: [id])
}

```

```

user      User      @relation(fields: [userId], references: [id])
transaction Transaction?

@@unique([lodgeId, userId, referenceYear, referenceMonth])
@@index([lodgeId])
@@map("invoices")
}

enum ExpenseStatus {
    PENDENTE_APROVACAO
    APROVADA // VM + Tesoureiro aprovaram
    PAGA
    REJEITADA
}

// Despesa – trava: exige aprovação dupla de VM e Tesoureiro
model Expense {
    id          String      @id @default(cuid())
    lodgeId     String
    description  String
    supplier     String?
    amountCents Int
    dueDate      DateTime?
    status       ExpenseStatus @default(PENDENTE_APROVACAO)

    approvedByMasterId String?
    approvedByMasterAt  DateTime?
    approvedByTreasurerId String?
    approvedByTreasurerAt DateTime?
    paidAt              DateTime?

    createdAt DateTime @default(now())
    updatedAt DateTime @updatedAt

    lodge      Lodge      @relation(fields: [lodgeId], references: [id])
    approvedByMaster User?  @relation("ExpenseMaster", fields: [approvedByMasterId], references: [id])
    approvedByTreasurer User? @relation("ExpenseTreasurer", fields: [approvedByTreasurerId], references: [id])
    transaction Transaction?

    @@index([lodgeId])
    @@map("expenses")
}

enum TransactionType {
    RECEITA
    DESPESA
}

// Livro-caixa consolidado – base dos balancetes mensais
model Transaction {
    id          String      @id @default(cuid())
    lodgeId     String
    type         TransactionType
    description  String
    amountCents Int
    date         DateTime
    category     String? // capitação, tronco, evento, aluguel...
    invoiceId    String?    @unique
    expenseId    String?    @unique
    donationId   String?    @unique
    createdAt    DateTime    @default(now())

    lodge      Lodge      @relation(fields: [lodgeId], references: [id])
    invoice     Invoice?   @relation(fields: [invoiceId], references: [id])
    expense     Expense?  @relation(fields: [expenseId], references: [id])
    donation    Donation? @relation(fields: [donationId], references: [id])

    @@index([lodgeId, date])
    @@map("transactions")
}

enum DonationType {
    TRONCO_SOLIDARIEDADE
    INGRESSO_EVENTO
    DOACAO_AVULSA
}

// Arrecadações extras: tronco e eventos beneficentes
model Donation {
    id          String      @id @default(cuid())
    lodgeId     String
    userId       String? // anônimo se null
    eventId      String?
    type         DonationType
    amountCents Int
    date         DateTime    @default(now())

    lodge      Lodge      @relation(fields: [lodgeId], references: [id])

```

```

user      User?      @relation(fields: [userId], references: [id])
event     CharityEvent? @relation(fields: [eventId], references: [id])
transaction Transaction?

@@index([lodgeId])
@@map("donations")
}

model CharityEvent {
  id          String   @id @default(cuid())
  lodgeId     String
  name        String
  date        DateTime
  ticketPriceCents Int?
  createdAt   DateTime @default(now())

  lodge       Lodge    @relation(fields: [lodgeId], references: [id])
  donations   Donation[]

  @@index([lodgeId])
  @@map("charity_events")
}

// ----- Módulo Secretaria: Kanban (loja.md §4) -----

enum StatusAdmissao {
  DOCUMENTACAO
  EDITAL_PUBLICADO
  SINDICANCIA
  ESCRUTINIO
  AGUARDANDO_PLACET
  INICIADO
  REPROVADO
}

// Pipeline de Admissão (Iniciação) – candidato profano até virar obreiro
model ProcessoAdmissao {
  id          String      @id @default(cuid())
  lodgeId     String
  nomeCandidato String
  cpf         String?
  email       String?
  phone       String?
  fotoUrl     String? // foto do candidato (data URI, até 500 KB)
  status      StatusAdmissao @default(DOCUMENTACAO)
  certidoesValidas Boolean @default(false)
  dataEscrutinio DateTime?
  aprovado    Boolean?
  observacoes String?
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt

  lodge       Lodge @relation(fields: [lodgeId], references: [id])

  @@index([lodgeId])
  @@map("processos_admissao")
}

enum StatusPlacet {
  PENDENTE
  EM_ANALISE
  APROVADO
  NEGADO
}

// Quite Placet (desligamento/filiação) – trava: exige Nada Consta da Tesouraria
// e dupla assinatura (VM + Secretário) para emissão final
model QuitePlacet {
  id          String      @id @default(cuid())
  userId      String
  lodgeId     String
  motivo      String?
  dataSolicitacao DateTime @default(now())
  quitacaoFinanceira Boolean @default(false)
  status      StatusPlacet @default(PENDENTE)

  signedByMasterId String?
  signedByMasterAt  DateTime?
  signedBySecId     String?
  signedBySecAt     DateTime?

  driveFileId String?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  user      User @relation(fields: [userId], references: [id])
  lodge     Lodge @relation(fields: [lodgeId], references: [id])
  signedByMaster User? @relation("QuitePlacetMaster", fields: [signedByMasterId], references: [id])
}

```

```

signedBySec    User? @relation("QuittePlacetSecretary", fields: [signedBySecId], references: [id])

@@index([lodgeId])
@@map("quitte_placets")
}

enum StatusProgressao {
    CUMPRIMENTO_INTERSTICIO
    INSTRUCAO_E_FREQUENCIA
    EXAME_PROFICIENCIA
    ESCRUTINIO_PROGRESSAO
    AGUARDANDO_PLACET
    AGUARDANDO_CERIMONIA
    COMUNICACAO_POS_CERIMONIA
    GRAU_CONCEDIDO
}

// Pipeline de Progressão de Graus (loja.md §4.B) – travas da Guarda dos
// Selos (placetDeferido) e da comunicação de 15 dias (comunicadoEnviado)
model ProcessoProgressao {
    id                String                @id @default(cuid())
    userId            String
    lodgeId           String
    grauAlvo          Degree // COMPANHEIRO ou MESTRE
    status            StatusProgressao @default(CUMPRIMENTO_INTERSTICIO)

    dataInicio        DateTime              @default(now())
    dataAprovacao     DateTime? // data que passou no escrutínio
    placetDeferido     Boolean              @default(false) // trava da Guarda dos Selos
    dataCerimonia      DateTime? // data da Elevação ou Exaltação
    comunicadoEnviado Boolean              @default(false) // trava dos 15 dias (Sua Sessão no GOB-SP)

    createdAt DateTime @default(now())
    updatedAt DateTime @updatedAt

    user User @relation(fields: [userId], references: [id])
    lodge Lodge @relation(fields: [lodgeId], references: [id])

    @@index([lodgeId])
    @@map("processos_progressao")
}

enum NotificationType {
    PENDING_SIGNATURE
    DEADLINE_WARNING
    MISSING_DATA
    FINANCIAL_APPROVAL
}

// Central de alertas (to-do de assinaturas, prazos de 15 dias, cadastro incompleto)
model Notification {
    id                String                @id @default(cuid())
    lodgeId           String
    userId            String?
    title             String
    description        String
    type              NotificationType
    isRead            Boolean              @default(false)
    dueDate           DateTime?
    createdAt         DateTime              @default(now())
    // Chave determinística da origem (ex.: "ata:<id>") – garante geração idempotente
    sourceKey         String?
    // Rota interna do recurso relacionado (ex.: "/secretaria/atas/<id>")
    link              String?

    lodge Lodge @relation(fields: [lodgeId], references: [id])
    user User? @relation(fields: [userId], references: [id])

    @@unique([lodgeId, sourceKey])
    @@index([lodgeId])
    @@map("notifications")
}

```